

Comparación de callbacks, promesas y async/await en el modelo de ejecución de JavaScript

Juan Velasco, Andrés Clavijo
Facultad de Ingeniería, Ingeniería Multimedia
Universidad de San Buenaventura, Bogotá, Colombia

Resumen—Este trabajo estudia el modelo de ejecución de JavaScript mediante un simulador de ocho avisos asincrónicos, escrito de tres formas distintas: callbacks anidados, promesas encadenadas y `async/await`. Cada aviso se guarda como un objeto con su identificador, su tipo, el instante para el que estaba programado y el instante en que realmente se atendió. Todos esos objetos se acumulan en un arreglo que después se procesa con métodos funcionales. Antes de ejecutar el programa se predijo el orden de salida usando la pila de ejecución y las colas de tareas, y luego se comparó esa predicción con lo que ocurrió de verdad. Se diseñó un caso límite propio, que deja la cola de macro tareas estancada mediante un bloqueo sincrónico de la pila. Los resultados muestran que las tres versiones producen exactamente los mismos valores, y que la diferencia entre ellas está en la legibilidad y en el manejo de errores. El caso límite muestra además que la cola de micro tareas se vacía por completo antes de atender cualquier macro tarea, incluso cuando esa macro tarea lleva 250 ms esperando.

Index Terms—event loop, JavaScript, promesas, `async/await`, callbacks, programación funcional, cola de micro tareas.

I. INTRODUCCIÓN Y OBJETIVO

JavaScript ejecuta el código en un solo hilo, coordinado por un bucle de eventos que reparte el trabajo entre una pila de llamadas, una cola de macro tareas y una cola de micro tareas [1]. Gracias a eso el programa puede esperar sin quedarse congelado, pero aparece una dificultad: el orden en que se escribe el código no siempre es el orden en que se ejecuta.

El objetivo de este laboratorio es poner en práctica ese modelo. Para eso se construyó un simulador que lanza ocho avisos con tiempos de espera distintos, escrito de tres maneras según la forma de expresar la espera. Después se predijo su comportamiento, se ejecutó y se compararon los resultados con la predicción.

El documento sigue el mismo orden del análisis. La Sección II describe la estructura común a las tres versiones. La Sección III las compara y concluye que hacen lo mismo, lo que permite que la Sección IV use una sola predicción de orden para las tres. Esa predicción explica qué significa la latencia que la Sección V calcula con métodos funcionales. El resultado principal de ese procesamiento es que el diseño secuencial impone el orden de atención sin importar el orden en que vencen las alarmas. La Sección VI rompe ese supuesto con un caso límite propio, y la Sección VII diagrama su momento clave.

II. ESTRUCTURA DEL SIMULADOR

Los ocho avisos se declaran en un módulo común, `eventsData.js`, para que las tres versiones partan de los mismos datos. Los tiempos se eligieron con dos parejas muy cercanas entre sí, 500/530 ms y 450/470 ms, para tener alarmas programadas casi al mismo tiempo.

Listing 1. Declaración de los ocho avisos (`Lab/eventsData.js`).

```
const EVENTS = [
  { name: 'eventOne', type: 'aviso largo', delay: 500 },
  { name: 'eventTwo', type: 'aviso largo', delay: 530 },
  { name: 'eventThree', type: 'aviso corto', delay: 300 },
  { name: 'eventFour', type: 'aviso corto', delay: 450,
    failsWhenSimulating: true },
  { name: 'eventFive', type: 'aviso corto', delay: 470 },
  { name: 'eventSix', type: 'aviso largo', delay: 1200 },
  { name: 'eventSeven', type: 'aviso largo', delay: 600 },
  { name: 'eventEight', type: 'aviso corto', delay: 150 },
];
```

Cuando un aviso se atiende, se crea un objeto con los cuatro datos que pide el enunciado: identificador (`eventName`), tipo (`eventType`), instante programado (`scheduledTime`) e instante real (`realTime`). Ese objeto se construye en un solo lugar, `eventsData.js`, que funciona como módulo de eventos: declara los ocho avisos y entrega las dos fábricas que los producen, una en estilo de callback y otra en estilo de promesa. Así, lo único que cambia entre las tres versiones es la forma de esperar, no la forma de armar el dato. Eso permite saber que cualquier diferencia que se observe viene del modelo de ejecución y no de cómo está escrito el código.

Listing 2. Construcción del objeto-evento, igual para los tres estilos (`Lab/eventsData.js`).

```
function buildNotice(config, referenceTime) {
  return {
    eventName: config.name,
    eventType: config.type,
    scheduledTime: referenceTime + config.delay,
    realTime: Date.now(),
  };
}
```

Conviene aclarar qué significa `scheduledTime`, porque de eso depende todo el análisis. Las ocho alarmas se programan en el mismo instante, `referenceTime`, cada una para un momento distinto. Por eso `scheduledTime` se calcula sobre ese instante común y no sobre el momento en que se arma cada temporizador. La resta `realTime - scheduledTime` no mide entonces el error del temporizador, sino cuánto se demoró el programa en atender la alarma: el tiempo entre el momento para el que estaba programada y

el momento en que quedó registrada. A esa medida se le llama latencia en el resto del documento.

La segunda decisión importante es que las tres versiones atienden los avisos uno tras otro: el aviso $n+1$ no se programa hasta que el aviso n termina. Esto viene de los tres estilos que pide el enunciado, que son encadenados por naturaleza. El arreglo `register` se llena entonces en el mismo orden en que se atienden los avisos, y el procesamiento del Paso 4 se apoya en eso.

III. COMPARACIÓN DE LOS TRES ESTILOS

Para comparar el manejo de errores con casos reales y no solo con teoría, el simulador acepta la bandera `--fail`, que hace fallar el cuarto aviso. Sin esa bandera las tres versiones completan el arreglo. Con ella, las tres cortan la cadena en el mismo punto, y se puede ver cómo reacciona cada estilo.

III-A. Manejo de errores

En **callbacks anidados** el error se propaga a mano y de forma repetitiva. Cada llamada recibe un segundo argumento con el manejador, y hay que pasarlo en los ocho niveles: `handleFailure` aparece ocho veces en `callback.js`. Si se olvida en un solo nivel, ese error no se reporta y la cadena se detiene en silencio. No hay nada en el lenguaje que obligue a pasarlo.

En **promesas encadenadas** el manejo queda en un solo sitio. Un `.catch` al final recoge el error que ocurra en cualquiera de los ocho `.then` anteriores, porque un rechazo sin atender pasa automáticamente a la siguiente promesa de la cadena hasta encontrar quien lo maneje. Se pasa de ocho manejadores a uno.

En **async/await** el manejo es el más fácil de leer: un `try/catch` normal envuelve todo el flujo, igual que en código sincrónico. Esto funciona porque `await` sobre una promesa rechazada lanza la excepción en el punto de espera [3]. Por dentro sigue siendo el mismo mecanismo de promesas [5].

Las tres versiones reaccionaron igual ante el fallo: la cadena se cortó en el cuarto aviso y el arreglo quedó solo con los tres primeros. Hay un detalle importante en ese resultado. En los tres casos, el procesamiento del Paso 4 se ejecutó sin ningún error sobre ese arreglo incompleto y entregó valores correctos: latencia promedio de 524,00 ms con tres avisos, en lugar de 1833,50 ms con ocho. Nada avisó que faltaban cinco avisos. Es decir, la pérdida de datos pasa desapercibida para quien usa el arreglo. Se vuelve sobre esto en la Sección VIII.

III-B. Legibilidad del orden de ejecución

Los callbacks anidados producen ocho niveles de indentación, y hay que leerlos de afuera hacia adentro para saber en qué orden ocurren las cosas. Las promesas encadenadas aplanan esa pirámide en una lista de `.then`, aunque pasar datos de un paso al siguiente todavía exige cuidado. En `async/await` el orden del texto coincide con el orden de ejecución, así que se lee de arriba hacia abajo sin tener que traducir nada mentalmente.

Tabla I
ARREGLO DE LA VERSIÓN CON PROMESAS ENCADENADAS. TIEMPOS CONTADOS DESDE `REFERENCE TIME`, EN MILLISEGUNDOS.

Aviso	Programado	Real	Latencia
<code>eventOne</code>	500	506	6
<code>eventTwo</code>	530	1036	506
<code>eventThree</code>	300	1349	1049
<code>eventFour</code>	450	1811	1361
<code>eventFive</code>	470	2292	1822
<code>eventSix</code>	1200	3498	2298
<code>eventSeven</code>	600	4111	3511
<code>eventEight</code>	150	4265	4115
Latencia promedio			1833,50

III-C. Recomendación técnica

Para backend asincrónico se recomienda usar `async/await` como estilo por defecto, con `try/catch` para los errores. Cuando las tareas no dependen entre sí, conviene lanzarlas juntas con `Promise.all` o `Promise.allSettled` en lugar de encadenarlas. La razón de esto es por el tiempo: en este mismo simulador, atender los ocho avisos uno tras otro tomó 4265 ms, cuando la espera más larga es de apenas 1200 ms. Encadenar sin necesidad multiplicó por más de tres el tiempo total.

IV. PREDICCIÓN Y COMPARACIÓN CON LO EJECUTADO

IV-A. Predicción antes de ejecutar

Como la sección anterior mostró que los tres estilos hacen lo mismo, se predice el mismo comportamiento para los tres. A continuación se explica el motivo en cada caso.

Callbacks anidados. Cada `setTimeout` se arma dentro del callback del anterior. La estructura queda vacía entre un aviso y el siguiente, y en la cola de macrotareas nunca hay dos temporizadores del simulador al mismo tiempo. El orden de salida será entonces 1-2-3-4-5-6-7-8.

Promesas encadenadas. Cada `.then` devuelve la promesa del siguiente aviso, así que el temporizador siguiente tampoco se arma hasta que el anterior termina. Se agrega un paso por la cola de microtareas en cada eslabón, pero ese paso ocurre entre dos macrotareas y no cambia el orden. Se predice el mismo orden 1-2-3-4-5-6-7-8.

async/await. El bucle `for...of` detiene la función en cada `await` y la continúa como microtarea cuando la promesa se resuelve. El resultado visible es igual al de las promesas encadenadas, porque `await` es solo una forma más cómoda de escribir el mismo mecanismo [2]. Se predice otra vez 1-2-3-4-5-6-7-8.

IV-B. Lo que ocurrió al ejecutar

La Tabla I muestra la ejecución de `promise.js`, con los tiempos en milisegundos contados desde `referenceTime`.

Los resultados confirman las tres predicciones. El orden fue 1-2-3-4-5-6-7-8, la latencia creció aviso tras aviso, `findFirstOutOfOrder` reportó `eventThree` y

Tabla II
RESULTADOS DE LAS TRES VERSIONES CON LOS MISMOS DATOS DE ENTRADA. EN LAS TRES, EL PRIMER AVISO CON DESVIACIÓN MAYOR A 50 MS FUE `eventTwo`.

Versión	Orden	Latencia media (ms)	Fuera de orden
Callbacks anidados	1-8	1834,25	<code>eventThree</code>
Promesas encadenadas	1-8	1833,50	<code>eventThree</code>
<code>async/await</code>	1-8	1848,88	<code>eventThree</code>

`findFirstDeviationAbove` con umbral de 50 ms reportó `eventTwo`. Las tres versiones coincidieron en todo, como muestra la Tabla II.

Hubo una sola diferencia con lo predicho, y fue de cantidad: la latencia promedio medida (entre 1833,50 y 1848,88 ms) quedó por encima de la teórica de 1798,75 ms, con un exceso de 35 a 50 ms. La causa es que `setTimeout` garantiza una espera *mínima*, no exacta: la macro tarea se encola cuando vence el temporizador, pero solo se atiende cuando la estructura/pila queda libre, y a eso se suma el tiempo de cálculo entre un aviso y otro [4]. Como los avisos van uno tras otro, ese pequeño exceso de cada aviso se acumula sobre todos los siguientes, y por eso la diferencia crece con el número del aviso en vez de mantenerse igual. Esa misma acumulación explica que la diferencia entre versiones llegue a 15 ms, más que el error de un solo temporizador. Por eso los valores exactos cambian de una ejecución a otra: lo que sí se repite siempre es el orden y la forma en que crecen las latencias.

V. PROCESAMIENTO DEL ARREGLO CON MÉTODOS FUNCIONALES

Todo el procesamiento se hizo con métodos funcionales de arreglos en `analysis.js`, sin bucles `for` tradicionales. Sobre el arreglo de la Tabla I los resultados fueron: latencia promedio 1833,50 ms; avisos con desviación mayor a 100 ms, de `eventTwo` a `eventEight`, o sea siete de ocho; primer aviso con desviación mayor a 50 ms, `eventTwo`; y primer aviso atendido fuera de orden, `eventThree`.

Listing 3. Selección de identificadores con `filter` y `map`

```
(lab/analysis.js).
1 function listIdentifiersDeviatingAbove(register, thresholdMs)
2 {
3   return toValidRegister(register)
4     .filter((notice) => deviationOf(notice) > thresholdMs)
5     .map((notice) => notice.eventName);
6 }
```

V-A. Por qué se eligió cada método

reduce para la latencia promedio, porque es la única de las cuatro operaciones que necesita reducir todos los elementos a un solo número. Ni `map` ni `filter` sirven para eso, porque los dos devuelven arreglos. Además `reduce` no modifica el arreglo original ni deja efectos colaterales, así que `register` se puede volver a usar tal cual en los otros tres procesamientos.

filter y luego **map** para la lista de identificadores, porque la tarea tiene dos partes distintas: primero escoger un subconjunto y después transformar cada elemento escogido. `filter` devuelve un arreglo nuevo con los elementos que

cumplen la condición, con el objeto completo. `map` lo transforma y deja solo `eventName`. Usar solo `map` no habría servido, porque `map` conserva el tamaño del arreglo y habría devuelto ocho identificadores en vez de siete. Usar solo `filter` tampoco, porque habría devuelto objetos completos y no la lista de identificadores que pide el enunciado.

find para las dos búsquedas, porque devuelve el primer elemento que cumple la condición y se detiene ahí mismo. Frente a `filter`, evita recorrer todo el arreglo para después botar todo menos el primer resultado. Además devuelve el objeto en sí, o `undefined` si no encuentra nada, que es más claro que recibir un arreglo cuando lo que se busca es un solo elemento.

forEach no se usó en ninguno de los cuatro procesamiento, porque siempre devuelve `undefined`. Habría obligado a crear variables por fuera (un acumulador, un arreglo destino, una bandera) para imitar lo que `reduce`, `filter`, `map` y `find` ya resuelven solos. Sí se usó `forEach` en otro lugar, al armar los temporizadores del caso límite (`edgeCase.js`, línea 44), y justo por el motivo contrario: ahí la operación es puro efecto colateral y no produce ningún valor que recolectar, así que `map` habría armado un arreglo de identificadores de temporizador que nadie usa.

El único bucle tradicional del proyecto es el `for...of` de `async.js`, y su uso está justificado: `await` tiene que detener la función en cada vuelta, y los métodos de arreglo no saben nada de promesas. Un `events.map(await ...)` no esperaría, sino que devolvería un arreglo de promesas sin resolver.

V-B. Qué significan los resultados

La latencia promedio de 1833,50 ms no mide el error del temporizador, que es de pocos milisegundos. Mide el costo de haber encadenado los avisos: cuánto espera en promedio una alarma desde que debía sonar hasta que el programa la atiende. En términos de backend es un tiempo de espera en cola, y su valor alto es la prueba numérica que respalda la recomendación de la Sección III de lanzar en paralelo las tareas independientes.

Sobre `findFirstOutOfOrder` hay que aclarar algo. Siempre reporta `eventThree`, sin variación, porque con un diseño secuencial el arreglo se llena siempre en el orden del código y la comparación depende solo de las esperas declaradas. Que el resultado sea siempre el mismo no significa que la función esté mal: es justamente lo que se quería mostrar, que el orden de atención lo impone la estructura del programa y no el orden en que vencen las alarmas. Ver qué pasa cuando se rompe ese supuesto es el objetivo del caso límite.

VI. DISEÑO DE UN CASO LÍMITE PROPIO

VI-A. Cómo se diseñó

El caso límite se llama *estancamiento de la cola de macro tareas (task starvation) por bloqueo sincrónico de la pila*. Reproduce un problema común en backend: una tarea de cálculo que ocupa el hilo mientras los temporizadores vencen sin poder atenderse.

El caso límite está en un archivo propio y reutiliza `analysis.js` del simulador, para que el caso base y el caso límite se midan con el mismo criterio y sus cifras se puedan comparar.

El escenario, en `Lab/edgeCase.js`, funciona así. Primero se arman tres alarmas al mismo tiempo, con esperas de 100, 200 y 300 ms (línea 44), lo que rompe el encadenamiento del simulador base. Enseguida se ejecuta una espera activa de 350 ms (línea 48), es decir un bucle que gira sin hacer nada y sin soltar la pila. Las tres alarmas vencen mientras la pila está ocupada. Al terminar el bloqueo se encola una microtarea que anuncia su final, `noticeD` (línea 50), programada para el instante 350 ms.

Listing 4. Parte central del caso límite (`Lab/edgeCase.js`).

```
1 CONCURRENT_NOTICES.forEach((config) => {
2   setTimeout(() => recordNotice(config), config.delay);
3 });
4
5 blockCallStack(BLOCK_DURATION_MS);
6
7 Promise.resolve().then(() => recordNotice(BLOCK_END_NOTICE))
  ;
```

El escenario no es trivial porque pone a prueba tres cosas al mismo tiempo: temporizadores ya vencidos que no pueden atenderse, competencia entre la cola de macrotareas y la de microtareas, y una inversión entre el orden en que se programan los avisos y el orden en que se atienden.

VI-B. Predicción

Se predice que: (1) durante el bloqueo no se atenderá ninguna alarma, aunque su momento ya haya pasado, porque una macrotarea solo se toma cuando la pila queda vacía; (2) al terminar el bloqueo, la cola de microtareas se vaciará antes que la de macrotareas, así que `noticeD` será el primero en registrarse aunque se haya programado de último; (3) las tres alarmas se atenderán después, en orden de vencimiento, y las cuatro tendrán casi el mismo `realTime`; (4) por eso la latencia *bajará* a medida que la espera sube, con valores cercanos a 250, 150 y 50 ms para 100, 200 y 300 ms, al revés de lo que pasa en el caso base; y (5) `findFirstOutOfOrder` devolverá `noticeA`, por ser el primer aviso atendido después de otro programado para más tarde.

VI-C. Ejecución y explicación

La ejecución confirmó las cinco predicciones. La Tabla III recoge el arreglo obtenido.

La explicación es la siguiente. Mientras `blockCallStack` ejecuta su espera activa, la pila nunca se vacía, así que el bucle de eventos no puede tomar ninguna tarea, aunque los tres temporizadores ya hayan vencido y sus callbacks estén encolados. Cuando la función termina y el módulo acaba, la pila queda libre y el motor aplica su regla principal: vaciar por completo la cola de microtareas antes de tomar la siguiente macrotarea [1]. Por eso `noticeD`, encolado como microtarea unos microsegundos antes, se adelanta a tres macrotareas que llevaban entre 50 y 250 ms esperando. Solo después se atienden `noticeA`, `noticeB`

Tabla III
ARREGLO DEL CASO LÍMITE. LAS FILAS ESTÁN EN EL ORDEN REAL DE ATENCIÓN. TIEMPOS CONTADOS DESDE `REFERENCE TIME`, EN MILISEGUNDOS.

Aviso	Prog.	Real	Latencia	Cola
<code>noticeD</code>	350	350	0	microtareas
<code>noticeA</code>	100	350	250	macrotareas
<code>noticeB</code>	200	350	150	macrotareas
<code>noticeC</code>	300	350	50	macrotareas
Latencia promedio		112,50		

y `noticeC`, en orden de vencimiento, que es el orden de llegada (FIFO) de la cola de macrotareas.

El efecto sobre el arreglo y sobre el procesamiento del Paso 4 se puede comprobar. El arreglo queda en el orden [D, A, B, C], que no coincide con el orden programado [A, B, C, D]. Por eso aquí `findFirstOutOfOrder` sí encuentra un resultado (`noticeA`), mientras que en el caso base solo confirmaba el efecto del encadenamiento. La latencia promedio baja a 112,50 ms, diez veces menos que en el caso base, lo que confirma que esa medida describe la demora en atender y no la precisión del temporizador. Y `filter` con `map` devuelve `noticeA` y `noticeB`, pero no `noticeC`: la alarma programada para más tarde es la que aparece como más puntual. Parece raro, pero es correcto, porque las cuatro se atendieron en el mismo instante.

VII. DIAGRAMA DEL MODELO DE EJECUCIÓN

La Figura 1 muestra los dos momentos clave del caso límite, e indica para cada elemento del diagrama la línea de `Lab/edgeCase.js` que lo produce. El panel (a) corresponde al momento en que las tres alarmas ya vencieron pero la pila sigue ocupada por la espera activa. El panel (b) corresponde al momento siguiente, cuando la pila se vacía y el motor tiene que decidir qué atender primero. La flecha marcada como primera es la que explica todo lo que se observó: la cola de microtareas tiene prioridad sobre la de macrotareas, sin importar cuánto lleve esperando cada una.

VIII. CONCLUSIONES

Las tres formas de escribir espera asincrónica dieron resultados iguales: mismo orden de atención, misma latencia promedio dentro del margen del temporizador y mismo aviso reportado fuera de orden. Esto confirma que las tres son el mismo mecanismo de promesas escrito con sintaxis distinta, y que elegir entre ellas es una cuestión de comodidad: manejo de errores en un solo `try/catch` frente a propagación manual repetida en ocho niveles, y una legibilidad del orden de ejecución que mejora de callbacks a promesas y de promesas a `async/await`.

El caso límite mostró que un bloqueo de 350 ms basta para dejar tres temporizadores vencidos sin atender, y que al liberarse la pila la cola de microtareas se vacía completa antes de tomar la primera macrotarea: `noticeD`, programado de último, se atendió primero. Ese comportamiento invierte la

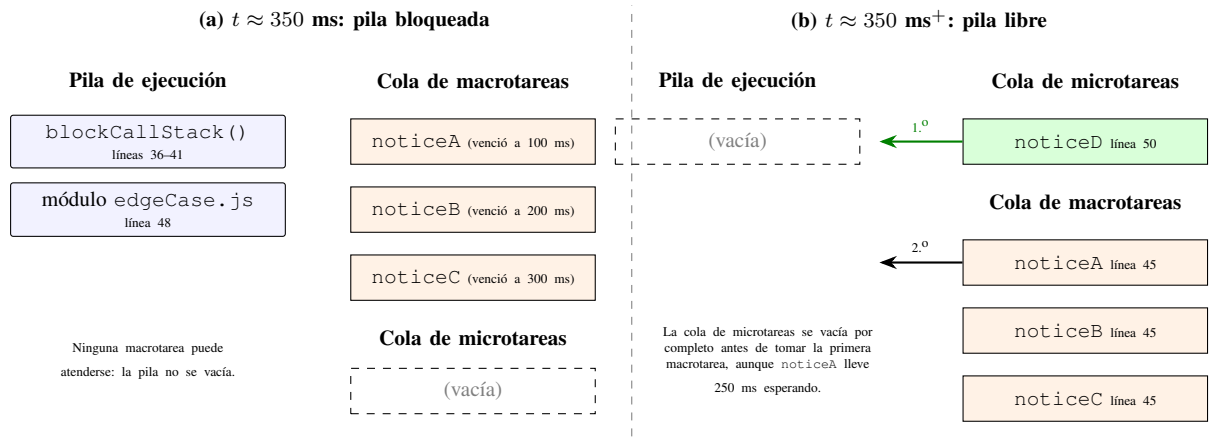


Figura 1. Pila de ejecución y colas de tareas en el caso límite. En (a) la espera activa de `blockCallStack` (Lab/edgeCase.js, líneas 36-41, llamada en la línea 48) mantiene ocupada la pila mientras los tres temporizadores armados en la línea 45 vencen y se acumulan en la cola de macro tareas. En (b) la pila queda libre: el motor vacía primero la cola de micro tareas, donde espera `noticeD` (línea 50), y solo después atiende las tres macro tareas en orden de vencimiento. Ese cambio de orden es el que registra la Tabla III.

tendencia de la latencia respecto al caso base y cambia los cuatro resultados del procesamiento funcional. Eso muestra que las medidas calculadas sobre el arreglo solo se pueden interpretar junto con el modelo de ejecución que las produjo.

Por último, la ejecución con fallo simulado dejó ver un problema de diseño del propio simulador: cuando la cadena se corta, `reduce`, `filter`, `map` y `find` trabajan sin error sobre un arreglo incompleto y devuelven valores que parecen correctos pero engañan. Una versión de producción debería registrar el fallo como un objeto-evento con un campo de estado, en lugar de omitirlo, para que la pérdida de datos se note.

REPOSITORIO

El código de las tres implementaciones, la estructura de objetos-evento y el procesamiento con métodos de arreglos están disponibles en: <https://github.com/Andryu18/Lab-DB>

REFERENCIAS

- [1] WHATWG, “HTML Living Standard — Section 8.1.7: Event loops,” 2024. [En línea]. Disponible en: <https://html.spec.whatwg.org/multipage/webappapis.html#event-loops>
- [2] Ecma International, *ECMAScript 2024 Language Specification*, 15.ª ed., Ginebra, Suiza: Ecma International, 2024.
- [3] Mozilla Developer Network, “await — JavaScript,” MDN Web Docs, 2024. [En línea]. Disponible en: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await>
- [4] OpenJS Foundation, “Timers | Node.js Documentation,” 2024. [En línea]. Disponible en: <https://nodejs.org/api/timers.html>
- [5] D. Flanagan, *JavaScript: The Definitive Guide*, 7.ª ed. Sebastopol, CA, EE.UU.: O’Reilly Media, 2020, cap. 13.